Test case 1: Retail Sales Performance Analysis

Using Google Colab, perform an analysis on a dataset of product sales to answer the following:
Dataset Columns: Product, Quantity, Price
Tasks:
1. Load the dataset into a Pandas DataFrame.
2. Fill missing values in the Price column with the average price of that product.
3. Add a new column: Total_Sales = Quantity × Price.
4. Identify the product with the highest total sales.
5. Visualize total sales by product using a bar chart
give me aim procedure , program and dataset

PROCEDURE

1. Open **Google Colab**.
2. Import required libraries: pandas, numpy, matplotlib.
3. Create or load the dataset into a Pandas DataFrame.
4. Check for missing values in the **Price** column.
5. Replace missing values with the **average price of the respective product**.
6. Create a new column:
    - Total_Sales = Quantity × Price
7. Group data by **Product** and calculate total sales.
8. Identify the product with the **highest total sales**.
9. Plot a **bar chart** to visualize total sales by product.
10. Analyze the output.

Program :

```
# Step 1: Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Step 2: Create dataset
data = {
   "Product": ["Laptop", "Mobile", "Laptop", "Tablet", "Mobile", "Tablet", "Laptop"],
   "Quantity": [5, 10, 3, 7, 8, 2, 6],
   "Price": [50000, 20000, None, 15000, None, 12000, 52000]
}

df = pd.DataFrame(data)

# Step 3: Fill missing Price values with product-wise average
df["Price"] = df.groupby("Product")["Price"].transform(lambda x: x.fillna(x.mean()))

# Step 4: Add Total_Sales column
df["Total_Sales"] = df["Quantity"] * df["Price"]

# Step 5: Group by Product and calculate total sales
sales_summary = df.groupby("Product")["Total_Sales"].sum()

# Step 6: Find product with highest sales
top_product = sales_summary.idxmax()
print("Product with highest total sales:", top_product)
```

```
# Step 7: Visualization
plt.figure()
sales_summary.plot(kind='bar')
plt.title("Total Sales by Product")
plt.xlabel("Product")
plt.ylabel("Total Sales")
plt.show()
```

Test case 2: City Temperature Monitoring

Using Google Colab, perform an analysis on a dataset of city temperatures to answer the following:
Dataset Columns: Date, Min_Temp, Max_Temp
Tasks:
1. Load the dataset and handle missing temperature values by filling them with column averages.
2. Add a new column: Average_Temp = (Min_Temp + Max_Temp) / 2.
3. Identify the date with the highest average temperature.
4. Visualize the average temperature trend using a line chart.

PROCEDURE

1. Open **Google Colab**.
2. Import required libraries: pandas, matplotlib.
3. Create or load dataset into a DataFrame.
4. Check missing values in Min_Temp and Max_Temp.
5. Replace missing values with **column average**.
6. Create new column:
    - Average_Temp = (Min_Temp + Max_Temp) / 2
7. Find the date with the **highest average temperature**.
8. Plot a **line chart** for temperature trend.
9. Analyze results.

PROGRAM :

```
import pandas as pd
import matplotlib.pyplot as plt

# Dataset
df = pd.DataFrame({
   "Date": ["2024-01-01", "2024-01-02", "2024-01-03", "2024-01-04", "2024-01-05"],
   "Min_Temp": [20, 22, None, 21, 23],
   "Max_Temp": [30, None, 35, 33, 36]
})

# FIXED: Fill missing values (no inplace)
df["Min_Temp"] = df["Min_Temp"].fillna(df["Min_Temp"].mean())
df["Max_Temp"] = df["Max_Temp"].fillna(df["Max_Temp"].mean())

# Average temperature
df["Average_Temp"] = (df["Min_Temp"] + df["Max_Temp"]) / 2

# Highest temperature date
max_date = df.loc[df["Average_Temp"].idxmax(), "Date"]
print("Highest Average Temperature Date:", max_date)

# Plot
```

```
plt.plot(df["Date"], df["Average_Temp"], marker='o')
plt.title("Average Temperature Trend")
plt.show()
```

Test case 3: Daily Traffic Volume Report

Using Google Colab, perform an analysis on a dataset of daily traffic volume to answer the following:
Dataset Columns: Date, Vehicle_Count
Tasks:
1. Load the dataset into a Pandas DataFrame.
2. Replace missing or null values in Vehicle_Count with 0.
3. Calculate total and average vehicle count.
4. Identify the date with the highest traffic volume.

Visualize daily traffic volume trend using a line graph.

PROCEDURE

1. Open **Google Colab**.
2. Import required libraries: pandas, matplotlib.
3. Create or load dataset into DataFrame.
4. Check for missing values in Vehicle_Count.
5. Replace missing values with **0**.
6. Calculate:
    - Total vehicle count
    - Average vehicle count
7. Find the date with the **highest traffic volume**.
8. Plot a **line chart** for daily traffic trend.
9. Interpret results.

PROGRAM:

```
import pandas as pd
import matplotlib.pyplot as plt

# Create dataset
df = pd.DataFrame({
   "Date": ["2024-01-01", "2024-01-02", "2024-01-03", "2024-01-04", "2024-01-05"],
   "Vehicle_Count": [120, 150, None, 180, 200]
})

# Replace missing values with 0
df["Vehicle_Count"] = df["Vehicle_Count"].fillna(0)

# Calculate total and average
total = df["Vehicle_Count"].sum()
average = df["Vehicle_Count"].mean()

print("Total Vehicle Count:", total)
print("Average Vehicle Count:", average)

# Find date with highest traffic
max_date = df.loc[df["Vehicle_Count"].idxmax(), "Date"]
print("Highest Traffic Date:", max_date)

# Plot line graph
plt.plot(df["Date"], df["Vehicle_Count"], marker='o')
plt.title("Daily Traffic Volume")
```

```
plt.xlabel("Date")
plt.ylabel("Vehicle Count")
plt.show()
```

Test case 4: E-Commerce Product Ratings Using Google Colab,

perform an analysis on a dataset of e-commerce product ratings to answer the following:

Dataset Columns: Product_Name, Customer_Rating

Tasks: 1. Load the dataset and remove rows with missing ratings.

2. Calculate the average customer rating.

3.Identify the top 3 highest-rated products.

4. Visualize the ratings for the top 5 products using a bar chart.

PROCEDURE

1. Open **Google Colab**.
2. Import required libraries: pandas, matplotlib.
3. Create or load dataset into a DataFrame.
4. Remove rows with missing Customer_Rating.
5. Calculate the **average rating**.
6. Identify **top 3 highest-rated products**.
7. Select top 5 products for visualization.
8. Plot a **bar chart** of ratings.
9. Analyze the results.

PROGRAM :

```
import pandas as pd
import matplotlib.pyplot as plt

# Create dataset
df = pd.DataFrame({
   "Product_Name": ["Laptop", "Mobile", "Tablet", "Headphones", "Camera", "Watch", "Speaker"],
   "Customer_Rating": [4.5, 4.2, None, 4.8, 4.6, 4.1, 4.7]
})

# Remove rows with missing ratings
df = df.dropna()

# Calculate average rating
avg_rating = df["Customer_Rating"].mean()
print("Average Rating:", avg_rating)

# Top 3 highest-rated products
top3 = df.sort_values(by="Customer_Rating", ascending=False).head(3)
print("\nTop 3 Products:\n", top3)

# Top 5 products for visualization
top5 = df.sort_values(by="Customer_Rating", ascending=False).head(5)

# Bar chart
plt.bar(top5["Product_Name"], top5["Customer_Rating"])
plt.title("Top 5 Product Ratings")
plt.xlabel("Product")
plt.ylabel("Rating")
plt.show()
```

Test case 5: Student Course Completion Report

Using Google Colab, perform an analysis on a dataset of student course completion status to answer the following:

Dataset Columns: Student_ID, Completion_Status (Yes/No)

Tasks:

1. Load the dataset into a DataFrame.
2. Fill missing values in Completion_Status with “No.”
3. Count how many students have completed and not completed the course.
4. Visualize completion vs non-completion using a pie chart.

PROCEDURE

1. Open **Google Colab**.
2. Import required libraries: pandas, matplotlib.
3. Create or load dataset into a DataFrame.
4. Replace missing values in Completion_Status with **"No"**.
5. Count number of students who:
    - Completed (Yes)
    - Not completed (No)
6. Plot a **pie chart** to visualize completion status.
7. Analyze results.

PROGRAM:

```
import pandas as pd
import matplotlib.pyplot as plt

# Create dataset
df = pd.DataFrame({
   "Student_ID": [101, 102, 103, 104, 105, 106, 107],
   "Completion_Status": ["Yes", "No", None, "Yes", "No", None, "Yes"]
})

# Fill missing values with "No"
df["Completion_Status"] = df["Completion_Status"].fillna("No")

# Count Yes and No
counts = df["Completion_Status"].value_counts()
print(counts)

# Pie chart
plt.pie(counts, labels=counts.index, autopct='%1.1f%%')
plt.title("Course Completion Status")
plt.show()
```

Code Link : https://colab.research.google.com/drive/10vpDoyWX9tFu9wnJ3GSiGVPIu70_cldS?usp=sharing